

# Software Verification

## Pragmatic BDD for Java

### (SB5-MAI)

Jan Corfixen Sørensen

University of Southern Denmark

# Behavior-Driven Development

# Why BDD?

Typical Test issues:

- ▶ Many technical and often irrelevant details
- ▶ Point of the test often hard to grasp
- ▶ Code duplication
- ▶ Can only be read by developers
- ▶ Cannot be used as documentation

# Behavior-Driven Development

- ▶ Behavior is described in a common domain language understandable by domain experts
- ▶ Domain experts and developers collaborate on defining the behavior
- ▶ Executed like normal tests
- ▶ Creates a living documentation

# BDD Example

Scenario: a pancake can be fried out of an egg milk and flour

Given an egg

And some milk

And the ingredient flour

When the cook mangles everything to a dough

And the cook fries the dough in a pan

Then the resulting meal is a pan cake

# “Classical” BDD Frameworks

- ▶ Cucumber: Plain Text + Java
- ▶ JBehave: Plain Text + Java
- ▶ Concordion: HTML + Java
- ▶ Fitness: Wiki + Java
- ▶ [robotframework.org](http://robotframework.org)

**Additional Maintenance Cost**

# Developer-friendly BDD Frameworks

- ▶ Spock: Groovy
- ▶ ScalaTest: Scala
- ▶ Jnario: Xtend
- ▶ Serenity\*: Java
- ▶ JGiven

JGiven



# Introduction

- ▶ Developer friendly (low maintenance overhead)
- ▶ Readable test code (Given-When-Then)
- ▶ Modular and reusable test code
- ▶ Reports for domain experts
- ▶ Open source <http://jgiven.org>

# Scenarios in JGiven

```
@Test
public void a_pancake_can_be_fried_out_of_an_egg_milk_and_flour() {
    given().an_egg().
        and().some_milk().
        and().the_ingredient( "flour" );

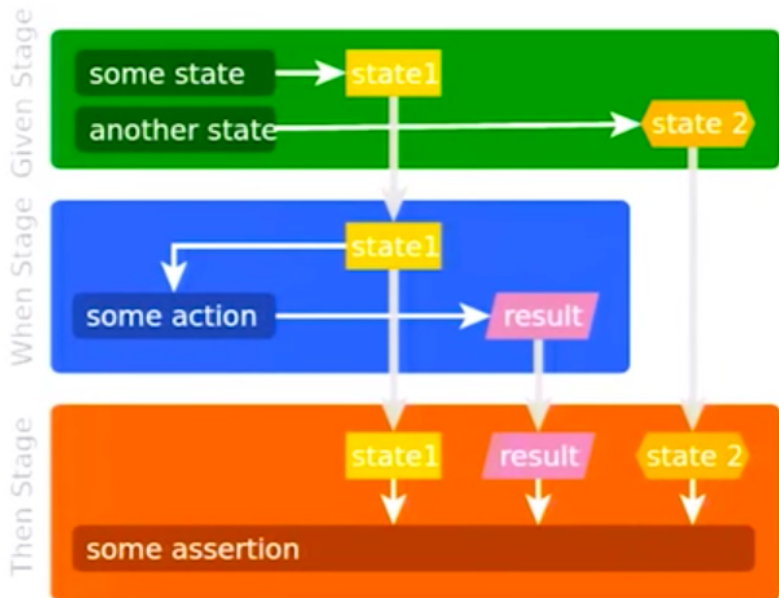
    when().the_cook_mangles_everything_to_a_dough().
        and().the_cook_fries_the_dough_in_a_pan();

    then().the_resulting_meal_is_a_pancake();
}
```

## Stage Classes: General

- JGiven scenarios are built from **stage classes**
- Stage classes provide **modularity** and **reuse**
- Stage classes are a unique feature of JGiven, not present in any other BDD framework
- Typically a stage class is used for either a Given, When, or Then steps of a scenario

# Stage Classes: State Transfer (1)



## Stage Classes: State Transfer (2)

- Fields can be annotated with **@ScenarioState**
- Values are written and read between stages
- **@ProvidedScenarioState**, **@ExpectedScenarioState** as alternative

# Stage Classes: Example (1)

```
public class GivenIngredients extends Stage<GivenIngredients> {

    @ProvidedScenarioState
    List<String> ingredients = new ArrayList<String>();

    public GivenIngredients an_egg() {
        return the_ingredient( "egg" );
    }

    public GivenIngredients the_ingredient( String ingredient ) {
        ingredients.add( ingredient );
        return this;
    }

    public GivenIngredients some_milk() {
        return the_ingredient( "milk" );
    }

}
```

## Stage Classes: Example (2)

```
public class WhenCook extends Stage<WhenCook> {  
    @Autowired  
    @ScenarioState  
    Cook cook;  
  
    @ExpectedScenarioState  
    List<String> ingredients;  
  
    @ProvidedScenarioState  
    Set<String> dough;  
  
    @ProvidedScenarioState  
    String meal;  
  
    public WhenCook the_cook_fries_the_dough_in_a_pan() {  
        assertThat( cook ).isNotNull();  
        assertThat( dough ).isNotNull();  
  
        meal = cook.fryDoughInAPan( dough );  
        return this;  
    }  
}
```

## Stage Classes: Example (3)

```
package com.tngtech.jgiven.examples.pancakes.test.steps;

import static org.assertj.core.api.Assertions.assertThat;

import com.tngtech.jgiven.Stage;
import com.tngtech.jgiven.annotation.ExpectedScenarioState;

public class ThenMeal extends Stage<ThenMeal> {
    @ExpectedScenarioState
    String meal;

    public void the_resulting_meal_is_a_pan_cake() {
        the_resulting_meal_is_a( expectedMeal: "pancake" );
    }

    public void the_resulting_meal_is_a( String expectedMeal ) {
         assertThat( meal ).isEqualTo( expectedMeal );
    }
}
```



# Console Output

```
Test Class: com.tngtech.jgiven.examples.pancakes.test.SpringPanCakeScenarioTest
```

```
A pancake can be fried out of an egg milk and flour
```

```
  Given an egg
```

```
    And some milk
```

```
    And the ingredient flour
```

```
  When the cook mangles everthing to a dough
```

```
    And the cook fries the dough in a pan
```

```
  Then the resulting meal is a pan cake
```

# HTML5 App

jGiven Report

ALL SCENARIOS

SUMMARY

All Scenarios 53

Failed Scenarios 0

Pending Scenarios 0

TAGS

BrowserTest

Features

Issue

CLASSES

com

## All Scenarios

53 Successful, 0 Failed, 0 Pending, 53 Total (9.077s)

Group By Sort By Status Tags Classes

1 2

|                              |                                                                              |            |                                                 |  |
|------------------------------|------------------------------------------------------------------------------|------------|-------------------------------------------------|--|
| > Argument Analyzer          | Different structure prevent data table                                       | ✓ (1ms)    | Issue-#103                                      |  |
| > Argument Analyzer          | Multiple formatted arguments lead to one parameter                           | ✓ (1ms)    | Issue-#103                                      |  |
| > Argument Analyzer          | Multiple parameter usages lead to one parameter                              | ✓ (2ms)    | Issue-#103                                      |  |
| > Ascii Doc Report Generator | The AsciiDoc reporter generates an index file                                | ✓ (12ms)   | AsciiDoc Report                                 |  |
| > Data Provider Test Ng      | A scenario with one failing case still executes the following ones           | ✓ (202ms)  | TestNG Issue-#103                               |  |
| > Data Provider              | A scenario with one failing case leads to a failed scenario                  | ✓ (3ms)    | JUnit                                           |  |
| > De Szenario                | Szenarien können in deutsch geschrieben werden                               | ✓ (2ms)    | German Scenarios                                |  |
| > Difference Analyzer        | The difference analyzer should find additional steps at the beginning        | ✓ (2ms)    | Case Diff                                       |  |
| > Difference Analyzer        | The difference analyzer should find additional steps at the end              | ✓ (1ms)    | Case Diff                                       |  |
| > Difference Analyzer        | The difference analyzer should find additional steps in the middle           | ✓ (17ms)   | Case Diff                                       |  |
| > Difference Analyzer        | The difference analyzer should find differences in step arguments            | ✓ (2ms)    | Case Diff                                       |  |
| > HTML App                   | Attachments appear in the HTML5 report                                       | ✓ (553ms)  | BrowserTest HTML5 Report Attachments            |  |
| > HTML App                   | Attachments of all cases appear in the HTML5 report when having a data table | ✓ (563ms)  | BrowserTest HTML5 Report Issue-#101 Attachments |  |
| > HTML App                   | Clicking on tag labels opens the tag page                                    | ✓ (1.372s) | BrowserTest HTML5 Report Tags Issue-#417        |  |
| > HTML App                   | Navigation links of the HTML report can be customized using a custom JS file | ✓ (1.243s) | BrowserTest HTML5 Report Issue-#145             |  |

# TNG Practical Experience

- ▶ 3 years of experience in a large Java Enterprise project (up to 70 developers)
- ▶ Over 3000 Scenarios
- ▶ Readability and reusability of test code has been greatly improved
- ▶ Maintenance costs of automated tests have been reduced (no hard numbers)
- ▶ Well accepted by developers
- ▶ Easy to learn by new developers
- ▶ Developers and domain experts collaborate using scenarios

# Summary

- ▶ Developer friendly
- ▶ Highly modular and reusable test code
- ▶ Just Java, no further language is needed
- ▶ Easy to integrate into existing test infrastructures (JUnit, TestNG)
- ▶ Open Source (Apache 2 Licence)
- ▶ Maven and Jenkins plugins
- ▶ Nice reports for domain experts
- ▶ Domain experts can not write scenarios in JGiven

AssertJ

# Motivation

JUnit's assertions underpowered from the start, Developers use frameworks like Hamcrest and Fest. Seeing a confusion of JUnit, Hamcrest and Fest.

- ▶ JUnit is very simplistic
- ▶ Fest looks to be abandonware
- ▶ Hamcrest is stagnant and ugly

# Why AssertJ?

- ▶ Still actively maintained
- ▶ Near complete superset of functionality
- ▶ Well designed
  - ▶ Easy to get started
  - ▶ Easy to enhance
  - ▶ Easy to read

# Basic Use

- ▶ `Assertions.assertThat()` type specific factory methods
- ▶ `AbstractAssert` type specific assertions
- ▶ Fluent API
- ▶ Good use of polymorphism

See more <http://joel-costigliola.github.io/assertj>



# Basic Use: Example

```
@Test
public void shouldProvideAnExample() {
    String actual = "This is a test";
    assertThat(actual).contains("is")
                      .startsWith("This");
    String[] actualArray = new String[]
        {"This", "is", "a", "test"};
    assertThat(actualArray).contains("is")
                           .startsWith("This");
}
```

- ▶ `assertThat` factory yields `String` or `Array` assert as needed
- ▶ `contains` and `startsWith` polymorphic
- ▶ fluent style i.e. `.contains().startsWith()`

# Custom Condition

- ▶ Custom conditions allow for more elaborate tests
- ▶ Can be applied to types that already have a custom assertion in straight forward way

# Custom Condition: Example

```
@Test
public void shouldBeEvenlyDivisibleBySix() {
    Condition<Integer> evenDivBySix = new
    Condition<Integer>() {
        @Override
        public boolean matches(Integer value) {
            return (value % 6) == 0;
        }
    };
    assertThat(12).is(evenDivBySix);
    assertThat(8).isNot(evenDivBySix);
}
```

- ▶ Implement a basic Condition functor
- ▶ Use your condition with is and isNot method

# Custom Assertion

Sometimes you've your own types with corresponding common assertions you'd like to apply to them. Perhaps you want to know if a `Student` instance `isInMiddleSchool`?

1. Subclass `AbstractAssert` for `Student` and implement a `isInMiddleSchool`.
2. Subclass `Assertions` adding a new `assertThat` factory method for your `StudentAssert` you created.
3. Use your new factory and assertions as you'd always (i.e. `assertThat(student).isInMiddleSchool()`).

# AssertJ Swing

- ▶ Simulation of user interaction with a GUI (e.g. drag 'n drop)
- ▶ Reliable GUI component lookup (by type, by name or custom search criteria)
- ▶ Support for all Swing components included in the JDK
- ▶ Compact and powerful API for creation and maintenance of functional GUI tests
- ▶ Ability to embed screenshots of failed GUI tests in HTML test reports
- ▶ Can be used with either TestNG or JUnit
- ▶ Supports testing violations of Swing's threading rules

# Getting Started

- ▶ To get quickly started, see [The one minute starting guide]
- ▶ If you have a bit more time, start with [The getting started guide]